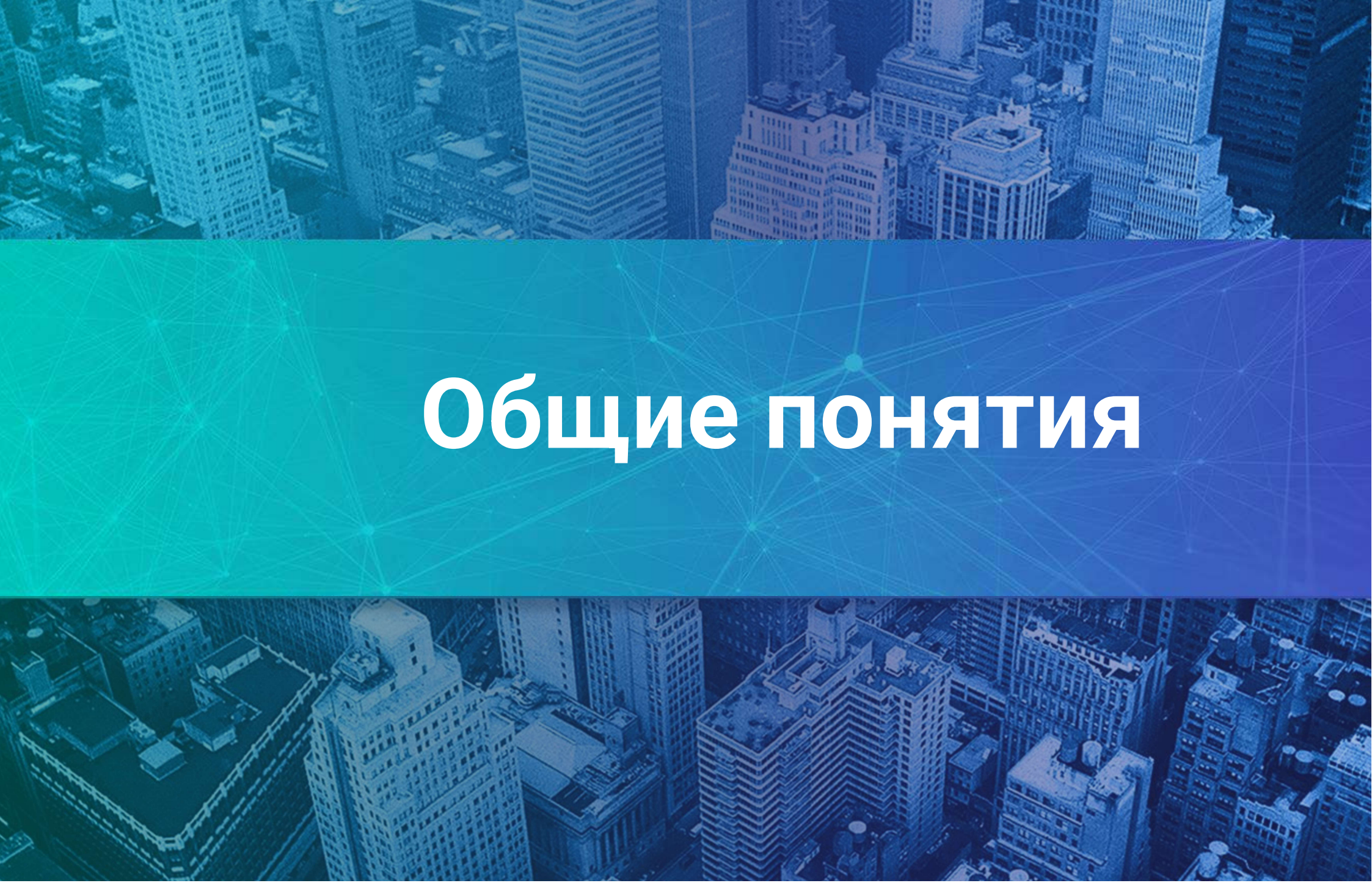


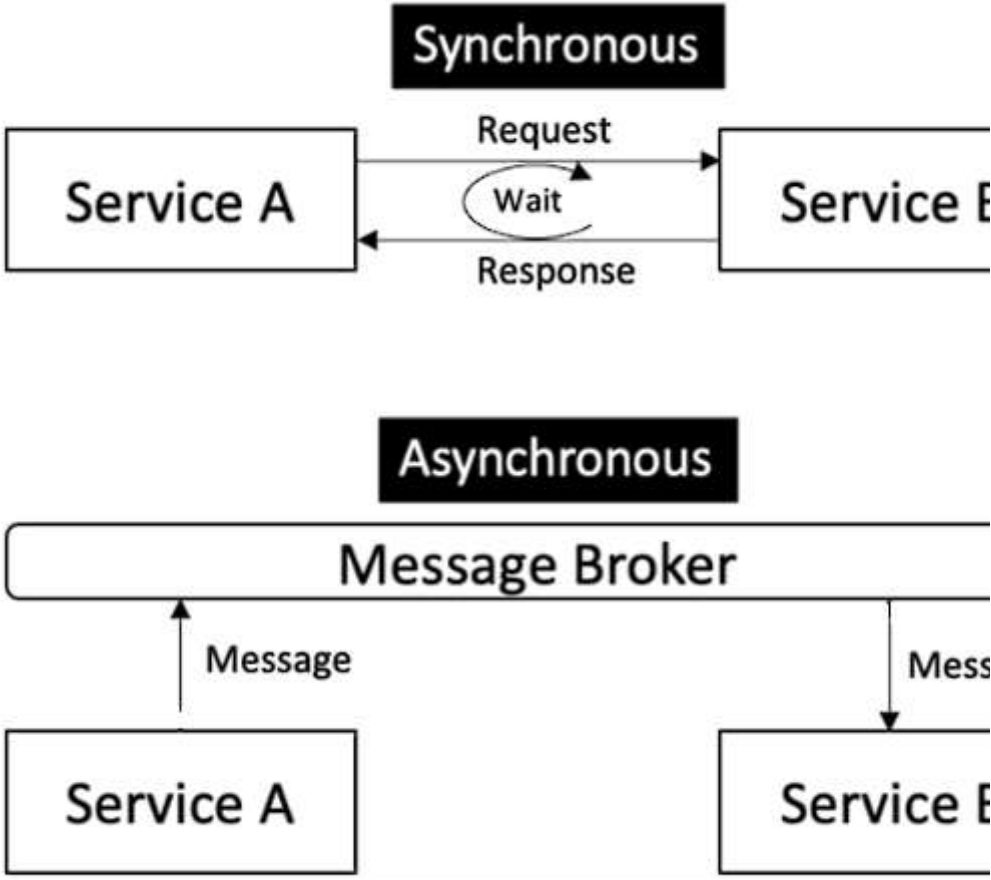
Общие понятия

Взаимодействия между микросервисами

Основные способы взаимодействий между сервисами
в микросервисной архитектуре:

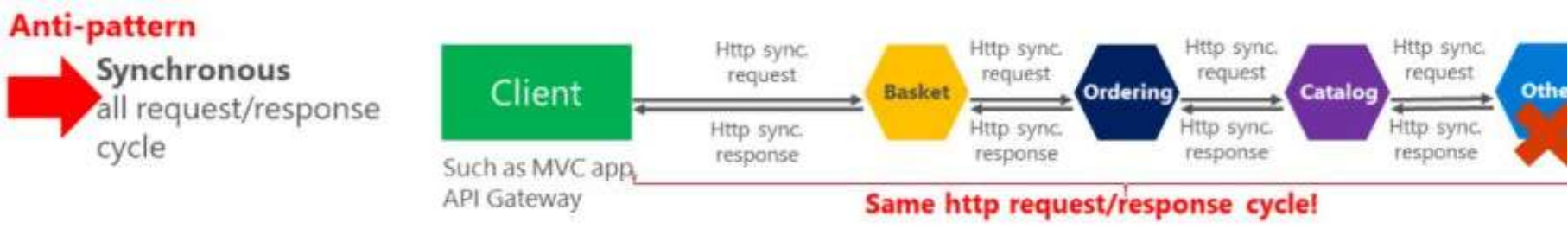
1. Синхронное взаимодействие
Отправитель ждет ответа на запрос
Например, HTTP

2. Асинхронное взаимодействие
Отправитель не ждет ответа на запрос.
AMQP (Advanced Message Queuing Protocol)

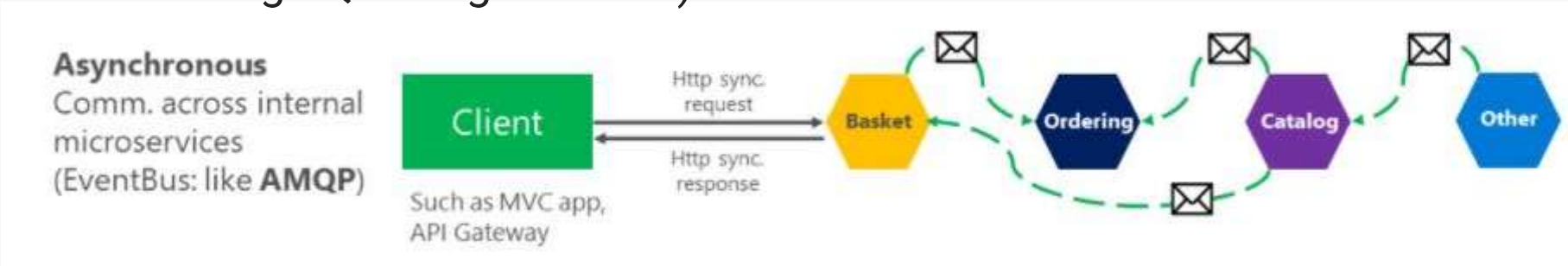


Взаимодействия между микросервисами

1. Синхронное взаимодействие
Отправитель ждет ответа на запрос
Например, HTTP

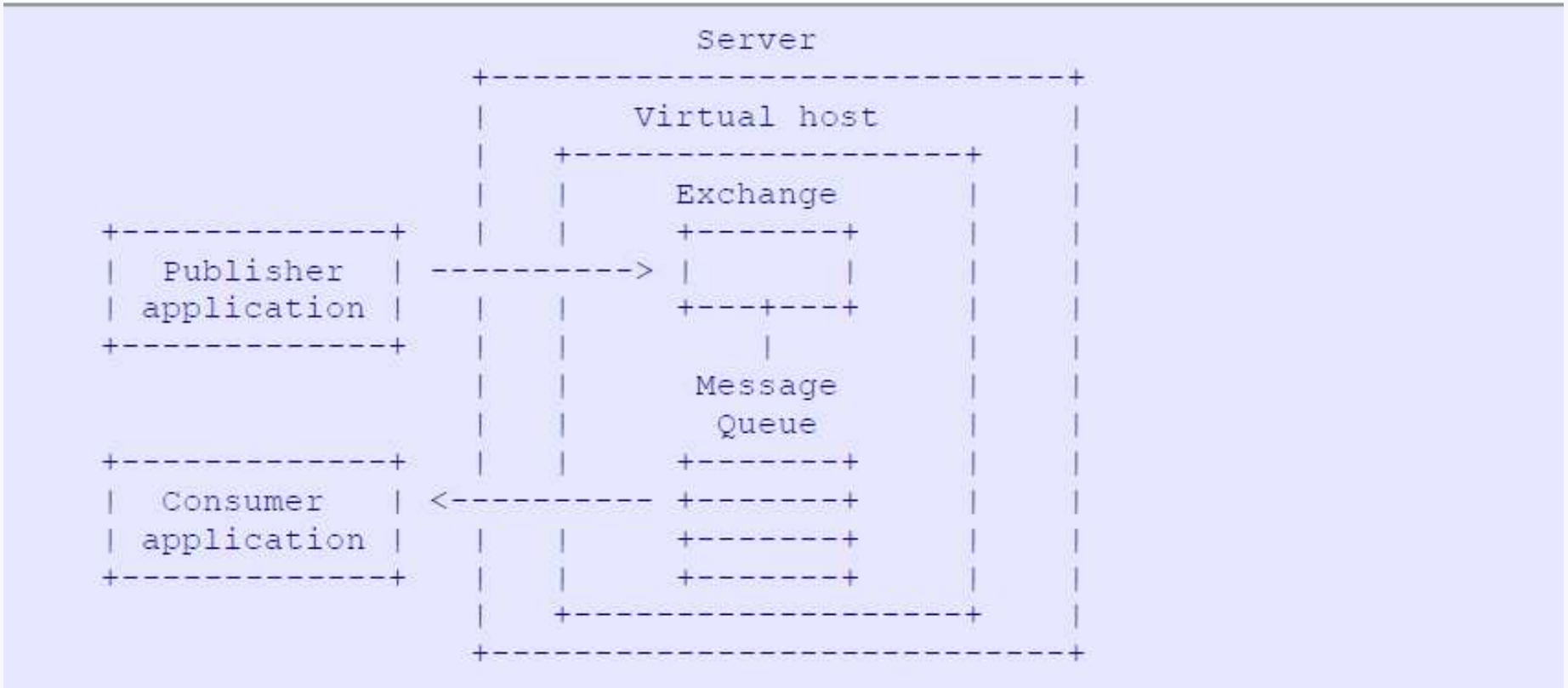


2. Асинхронное взаимодействие
Отправитель не ждет ответа на запрос.
AMQP (Advanced Message Queuing Protocol)



Брокер сообщений

Сервис, который принимает сообщения от клиента и осуществляет их маршрутизацию постановку в очередь.



https://www.rabbitmq.com/resources/specs/amqp0-9-1.pdf

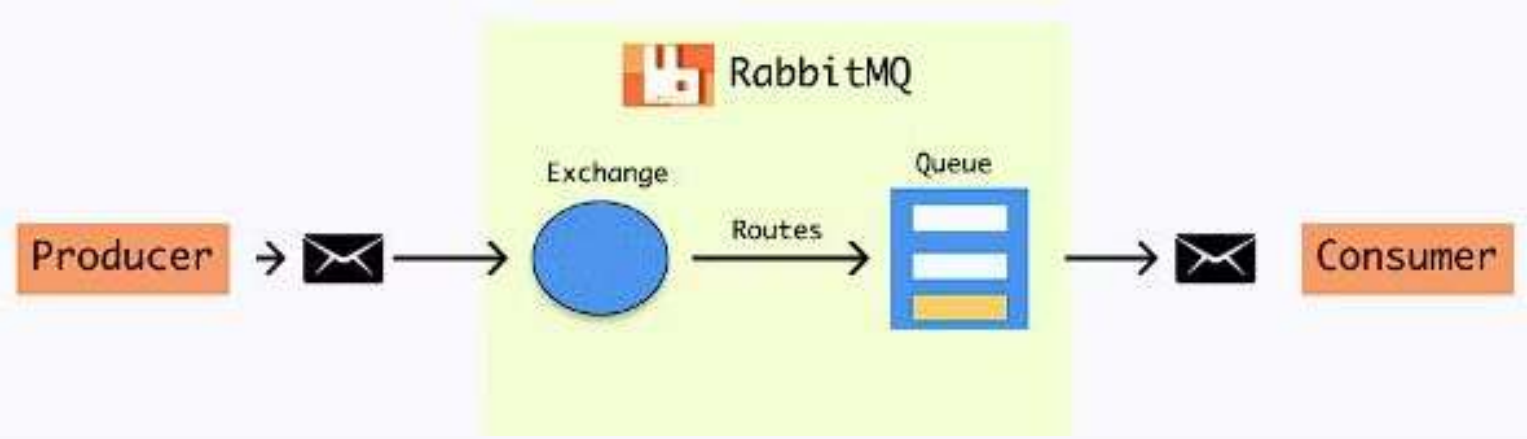
AMQP

AMQP – протокол, позволяющий приложениям обмениваться сообщениями через специал
сервисы – посредники, называемые брокерами сообщений.

(https://www.rabbitmq.com/tutorials/amqp-concepts.html)

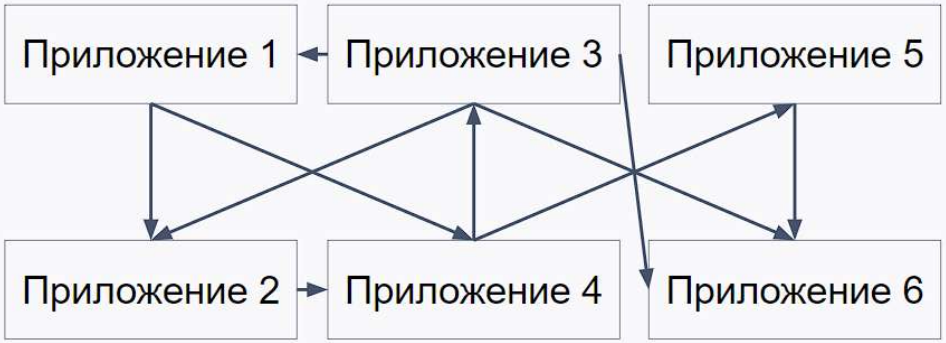
Основные функции брокера

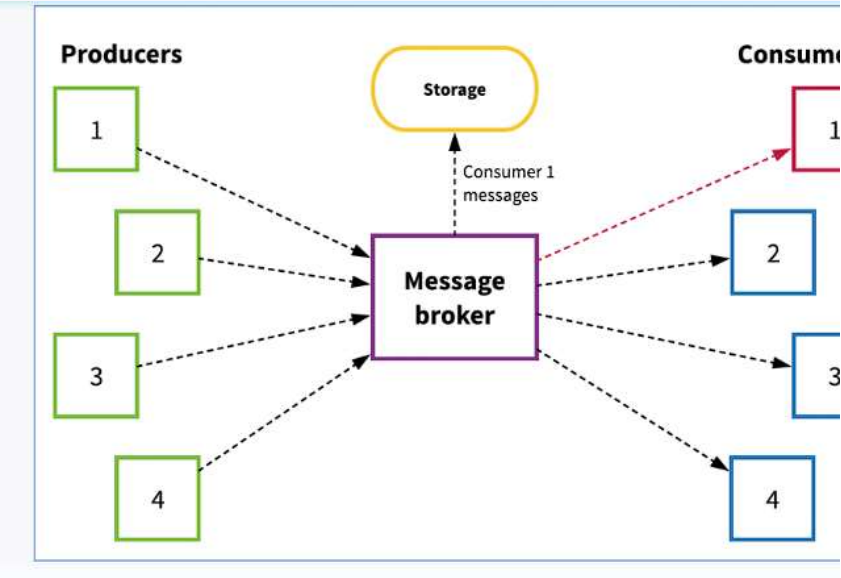
1. Обеспечение возможности асинхронной обработки сообщений, достигающееся нал
очередей



Основные функции брокера

2. Централизованная маршрутизация





Некоторые реализации брокеров сообщений

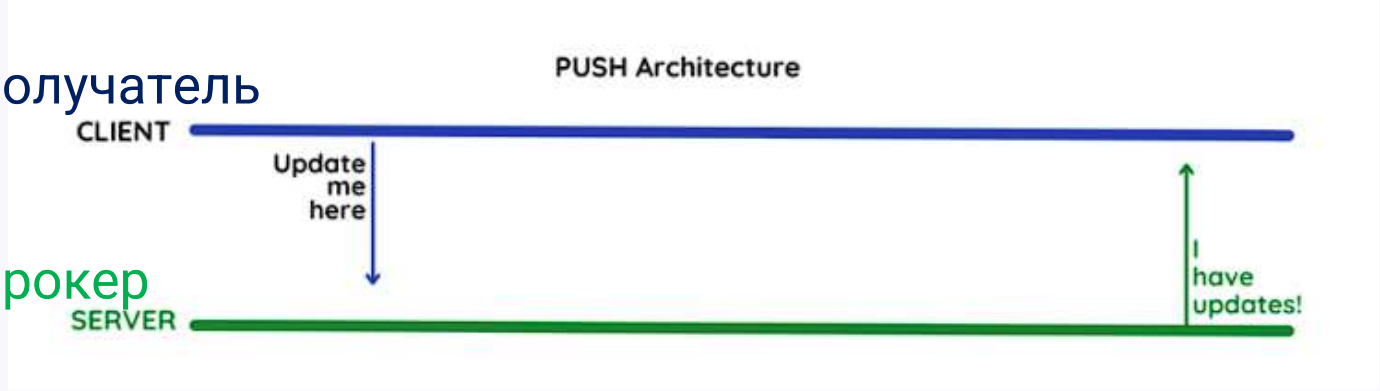
1.RabbitMQ 

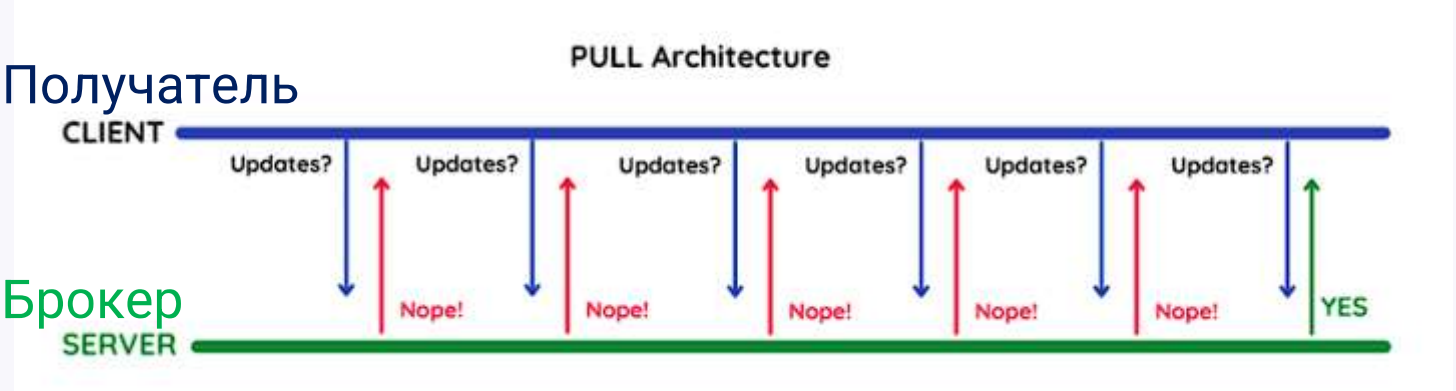
2.Kafka 

3.ZeroMQ 

…

PUSH и PULL архитектуры брокеров





Клиентские библиотеки для работы с брокера



Библиотеки для работы в .Net

- RabbitMQ.Client
- Masstransit.RabbitMQ
- NServiceBus
- EasyNetQ
- …

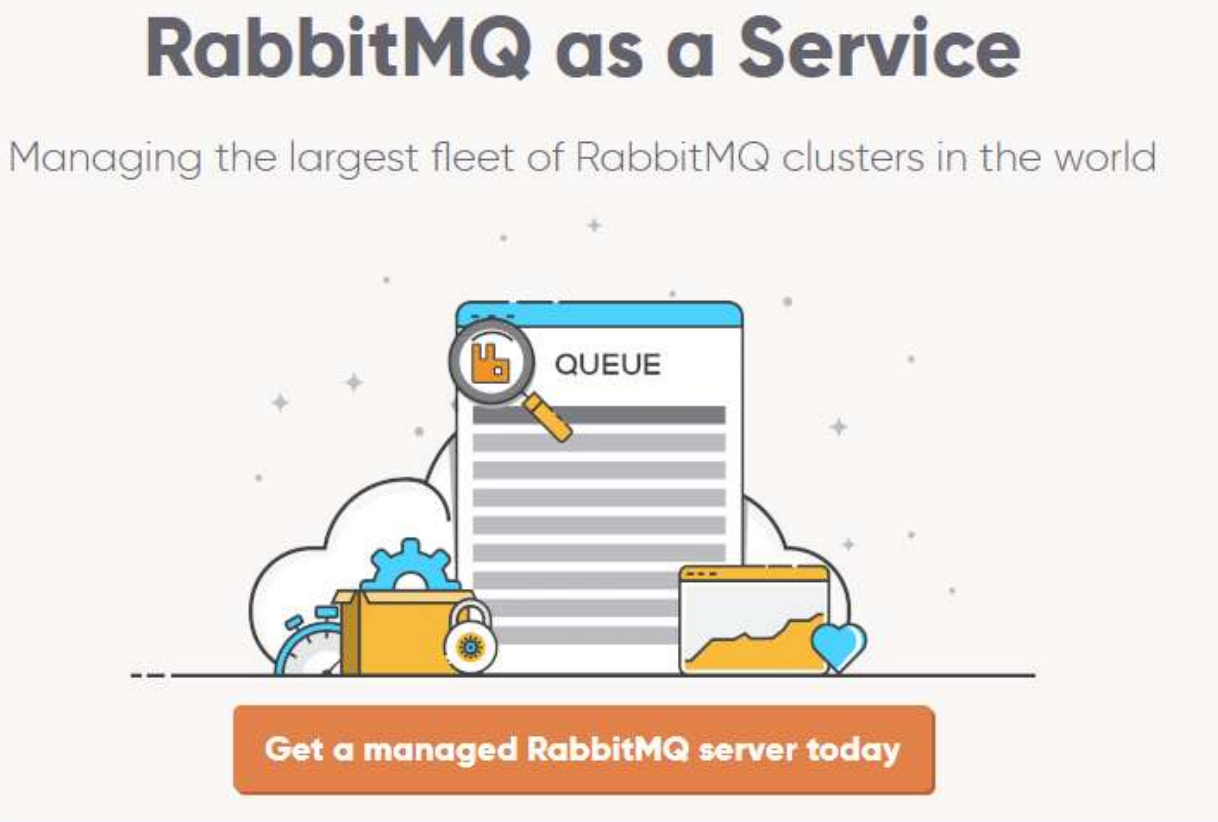


- Confluent Kafka Client
- …


RabbitMQ

Как будем использовать

https://www.cloudamqp.com/



Подключение помощью RabbitMQ.Client



Nuget: **RabbitMQ.Client**

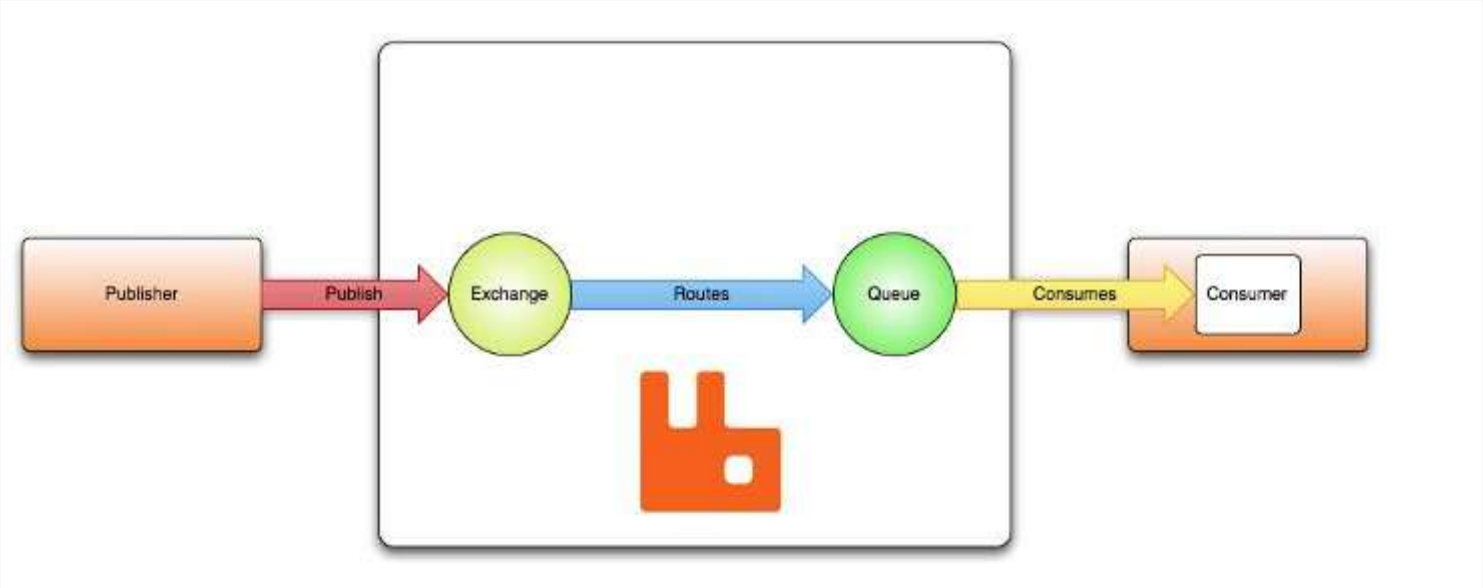
```
var cf = new ConnectionFactory();
var conn = cf.newConnection();

// the .NET client calls channels "models"
var ch = conn.CreateModel();
// do some work

// close the channel when it is no longer needed

ch.Close();
```

Основные понятия



Publisher (отправитель) – сервис, отправляющий сообщение

Consumer (потребитель) – сервис, принимающий сообщение

Exchange (обменник) – маршрутизатор, определяющий, в какую очередь должно б отправлено сообщение

Queue (очередь) – место, где хранятся уже отправленные но еще не принятые сообщения принципу FIFO

Адресно-маршрутная информация сообщения

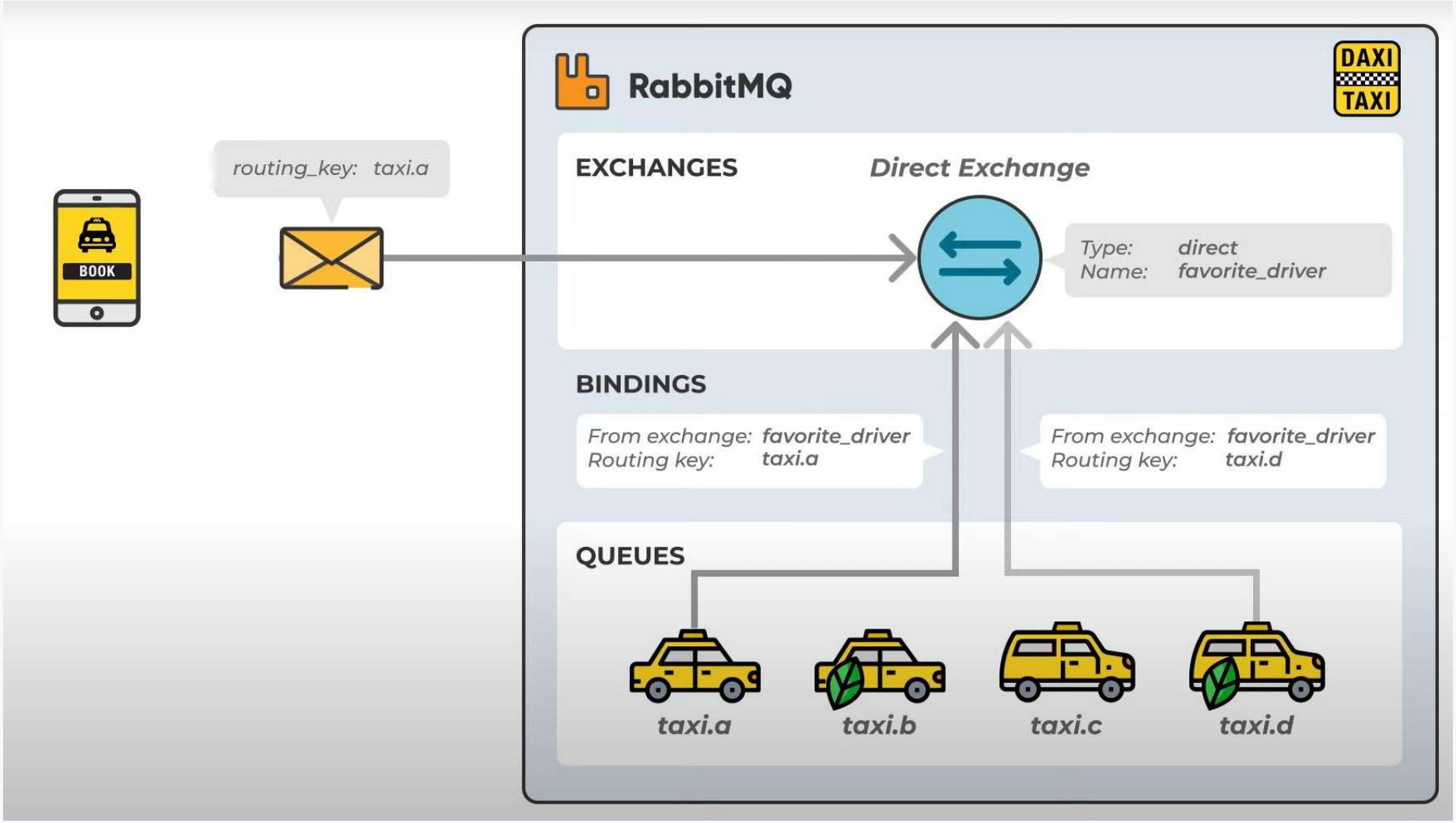
1. Наименование обменника (exchange)
2. Маршрутный ключ (routing key)
3. Хедеры (headers)

```
channel.BasicPublish(exchange: exchangeName,
    routingKey: "cars.2",
    basicProperties: null,
    body:  body);
```

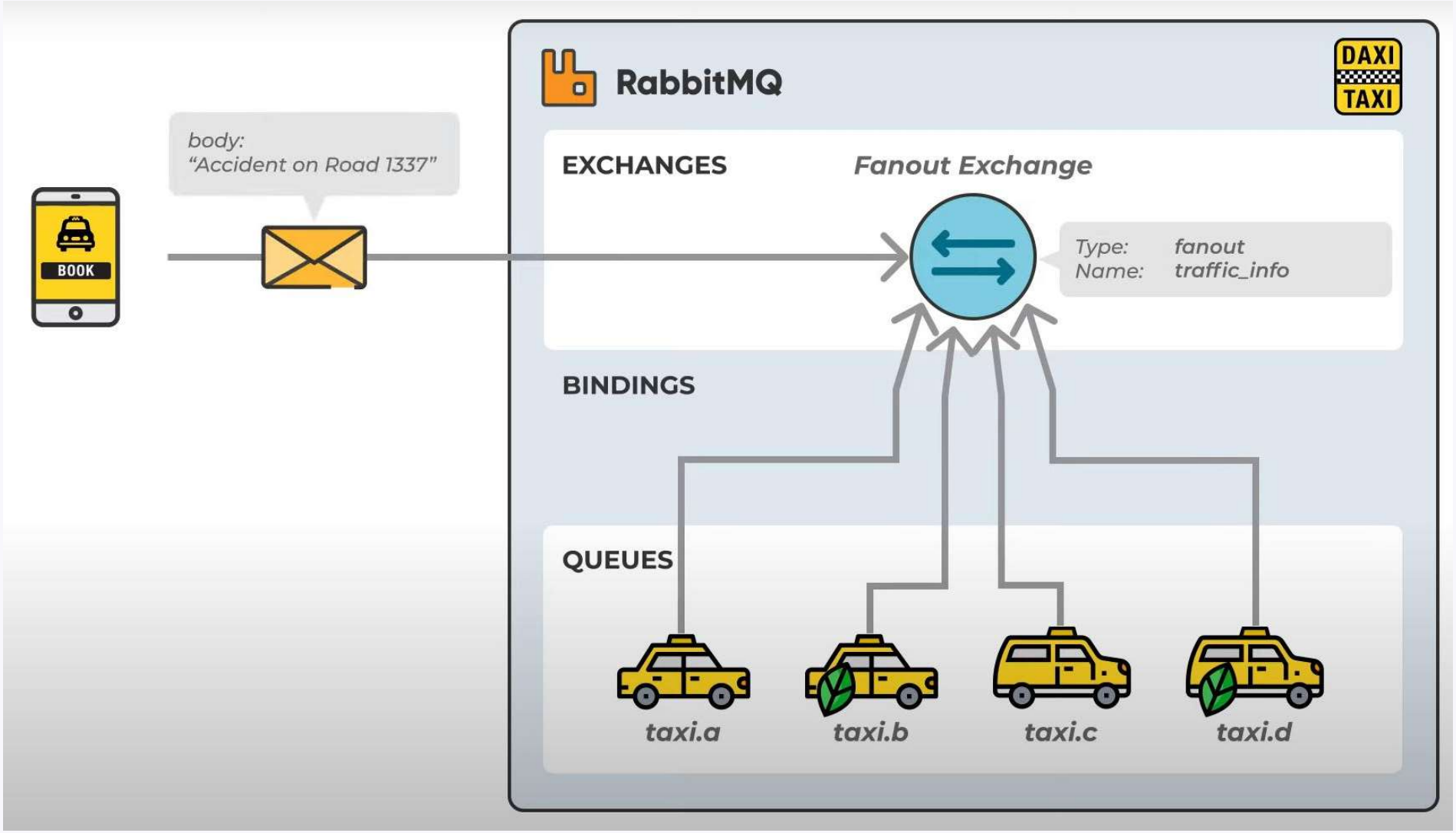
Виды Exchange

1. Direct
2. Topic
3. Fanout
4. Headers

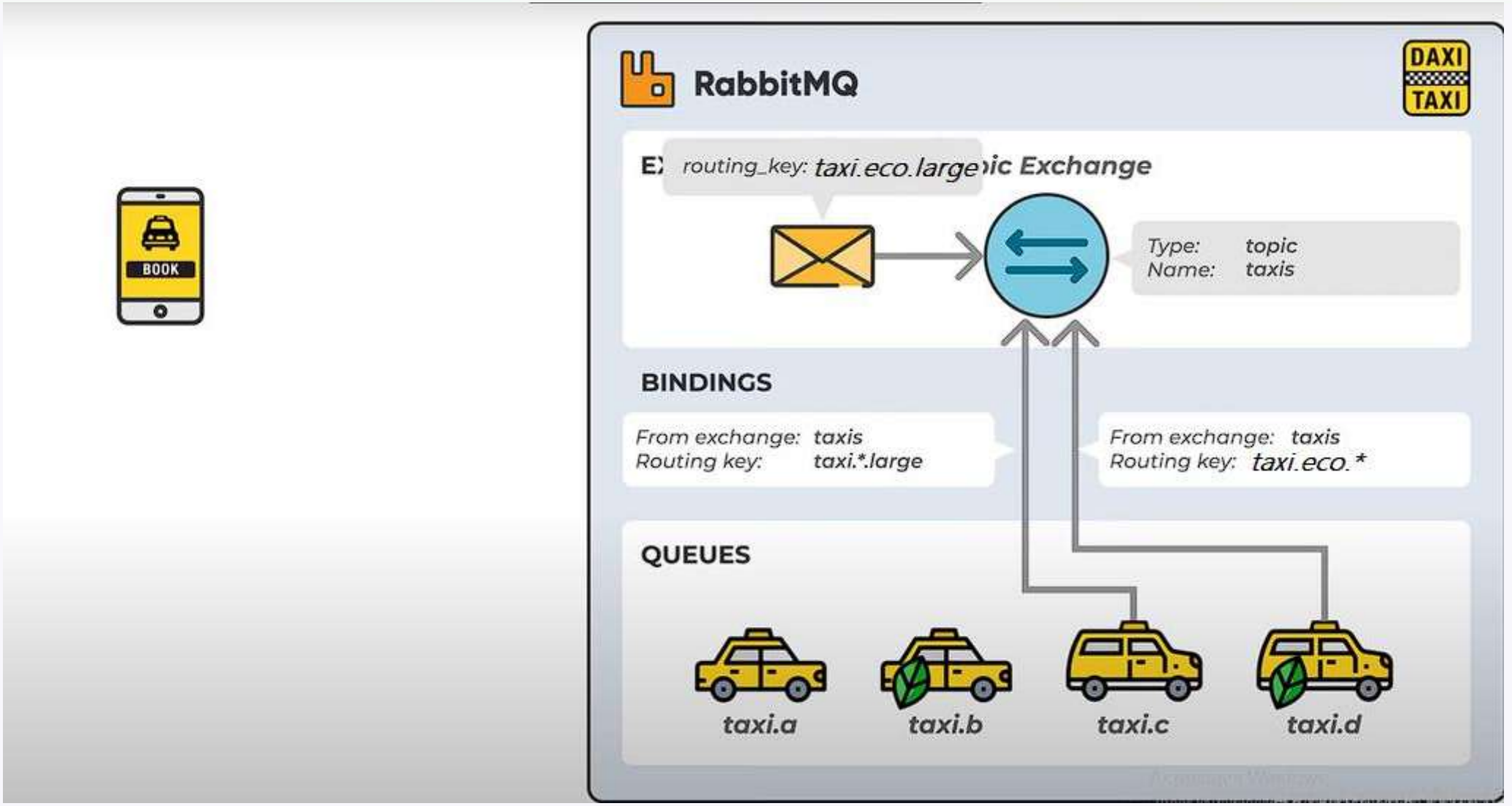
Direct Exchange



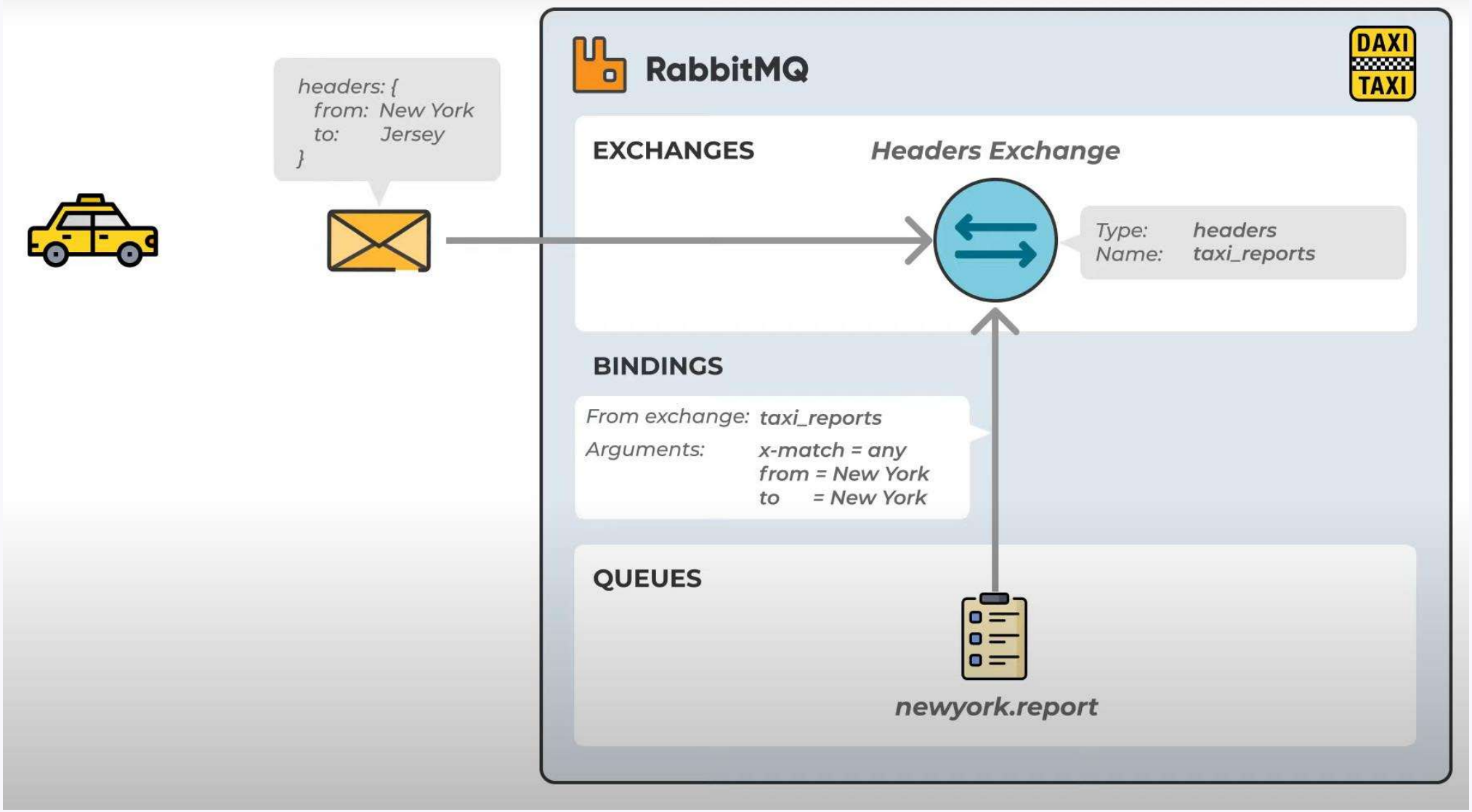
Fanout Exchange



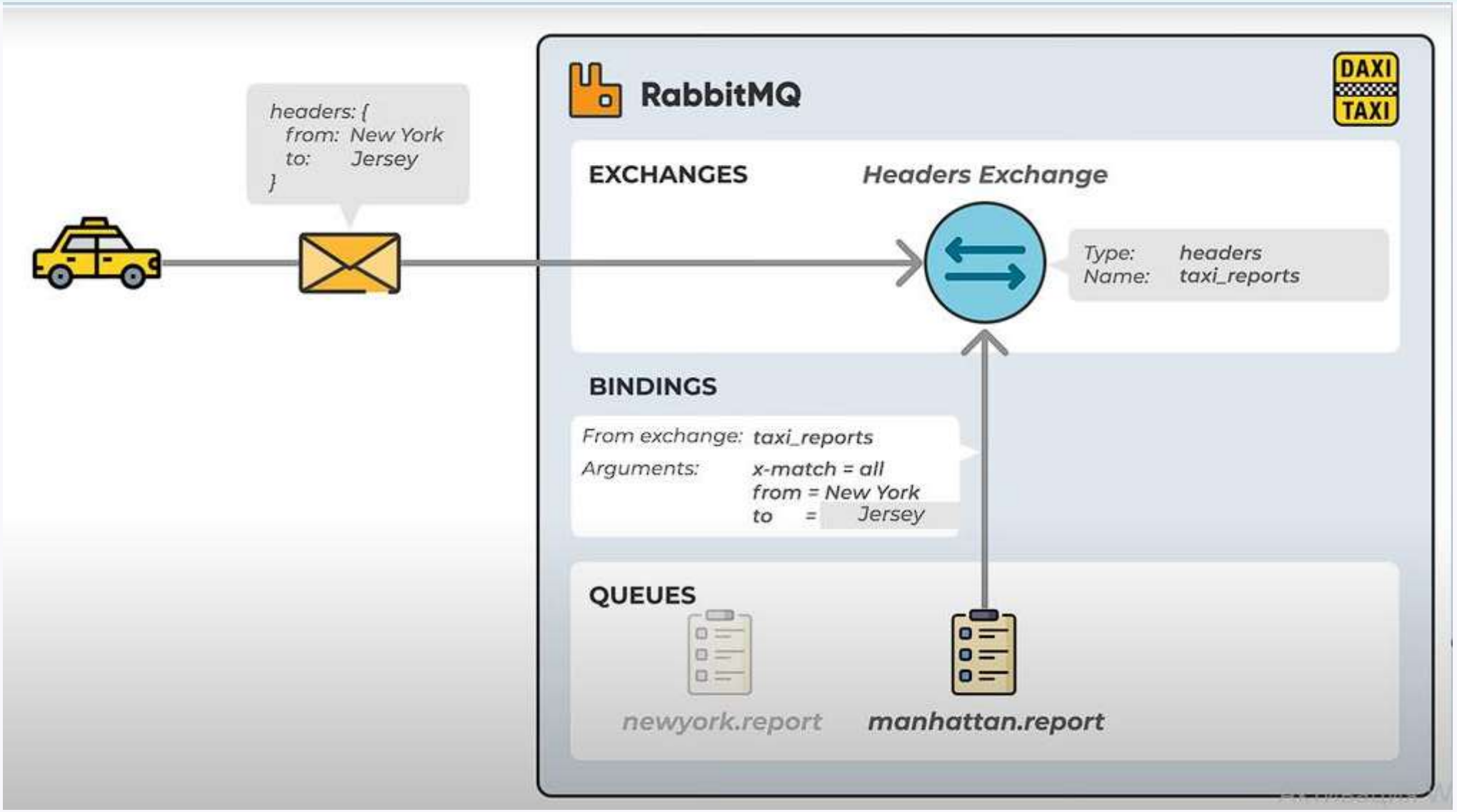
Topic Exchange



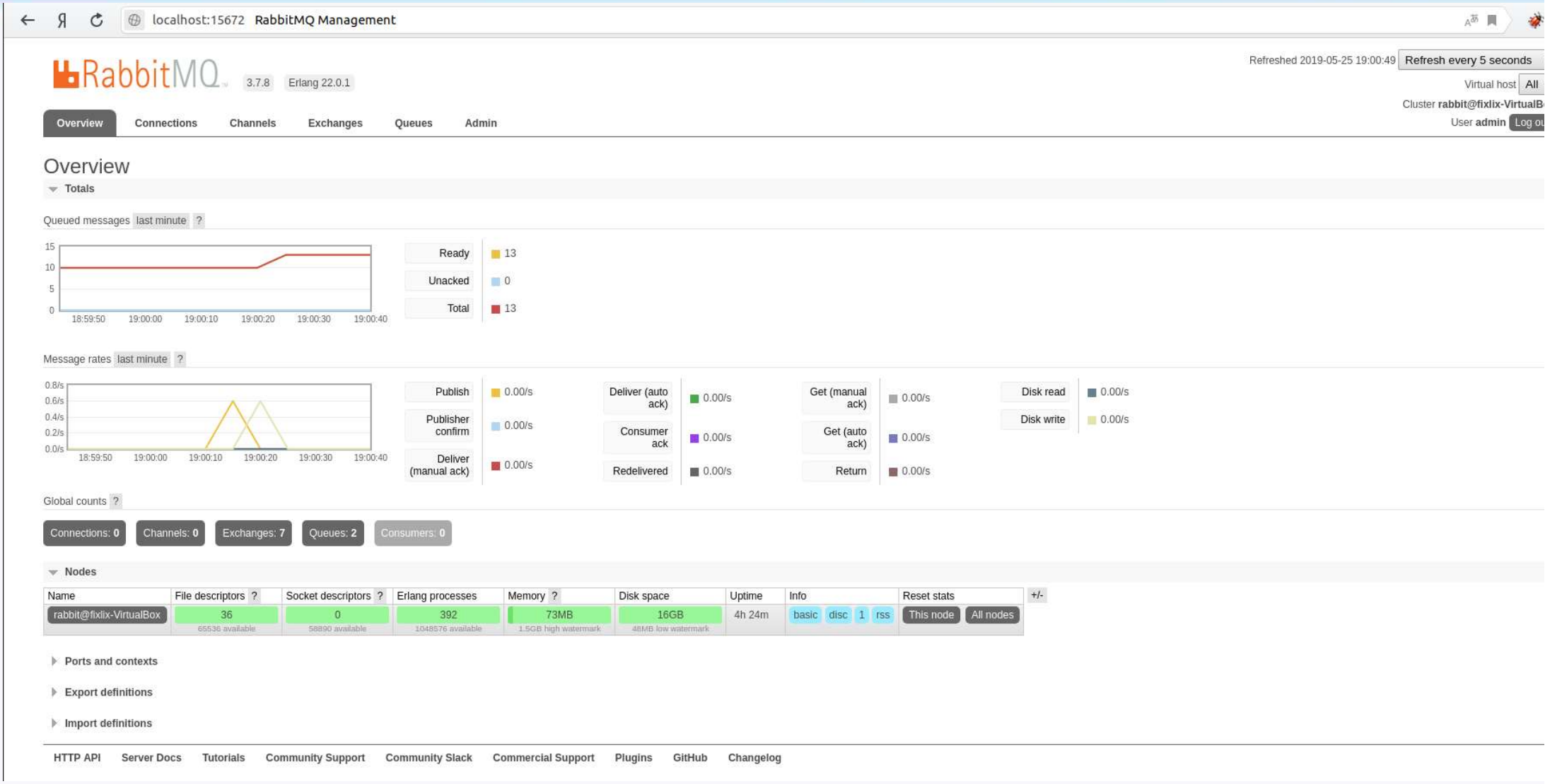
Headers Exchange



Headers Exchange



Панель администрирования



Настройки количества неподтвержденных сообщений в очереди

PrefetchSize – объем (в байтах) сообщений

PrefetchCount – количество сообщений

Global – применяется ли ко всем консьюмерам или только к данному

Настройки очереди

```
/// <summary>
/// Declares a queue. See the <a href="https://www.rabbitmq.com/queues.html">Queues guide</a> to learn more.
/// </summary>
/// <param name="queue">The name of the queue. Pass an empty string to make the server generate a name.</param>
/// <param name="durable">Should this queue will survive a broker restart?</param>
/// <param name="exclusive">Should this queue use be limited to its declaring connection? Such a queue will be deleted when its declaring
/// <param name="autoDelete">Should this queue be auto-deleted when its last consumer (if any) unsubscribes?</param>
/// <param name="arguments">Optional; additional queue arguments, e.g. "x-queue-type"</param>
[AmqpMethodDoNotImplement( namespaceName: null)]
 IL code
QueueDeclareOk QueueDeclare(string queue, bool durable, bool exclusive,
    bool autoDelete, IDictionary<string, object> arguments);
```

Durable – должна ли очередь сохраниться после перезапуска брокера

Exclusive – должна ли очередь удалиться после закрытия коннекта

Autodelete – должна ли очередь удалиться после отписки от нее консьюмера

Настройка контейнера с Rabbit

https://hub.docker.com/_/rabbitmq

1. Добавить в docker-compose в блок services:

```
rabbit:
  image: rabbitmq:3-management
  restart: always
  hostname: rabbitmqhost
  environment:
    RABBITMQ_DEFAULT_USER: guest
    RABBITMQ_DEFAULT_PASS: guest
  volumes:
    - rabbitmq_data:/var/lib/rabbitmq
  ports:
    - "5672:5672"
    - "15672:15672"
```

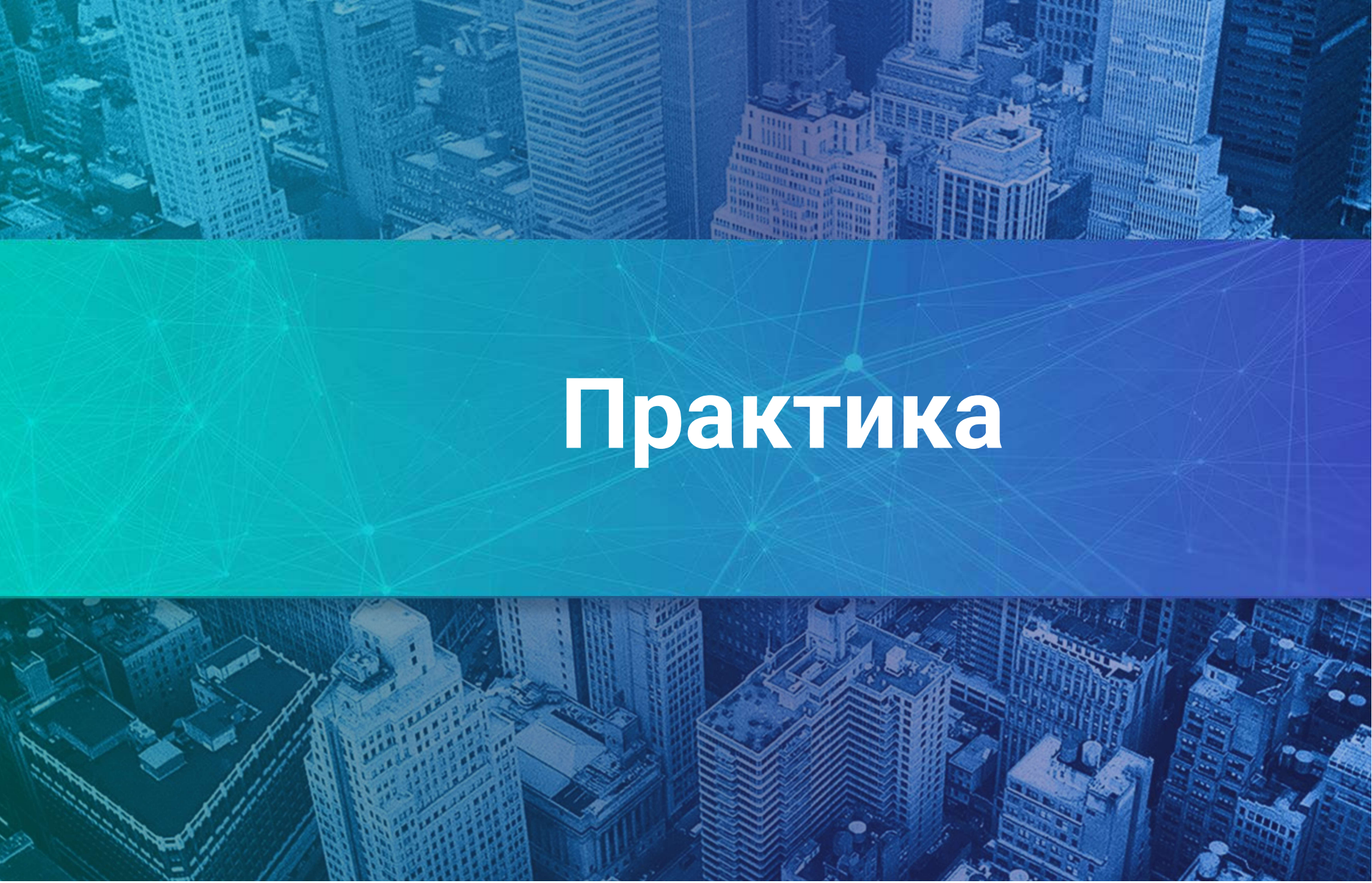
2. Добавить в docker-compose в блок volumes:

```
rabbitmq_data:
```

3. Добавить в данные подключения в appsettings.json, например:

```
"RmqSettings": {
  "Host": "localhost",
  "VHost": "/",
  "Login": "guest",
  "Password": "guest"
},
```

4. Считать данные из пункта 3 при инициализации клиентской библиотеки

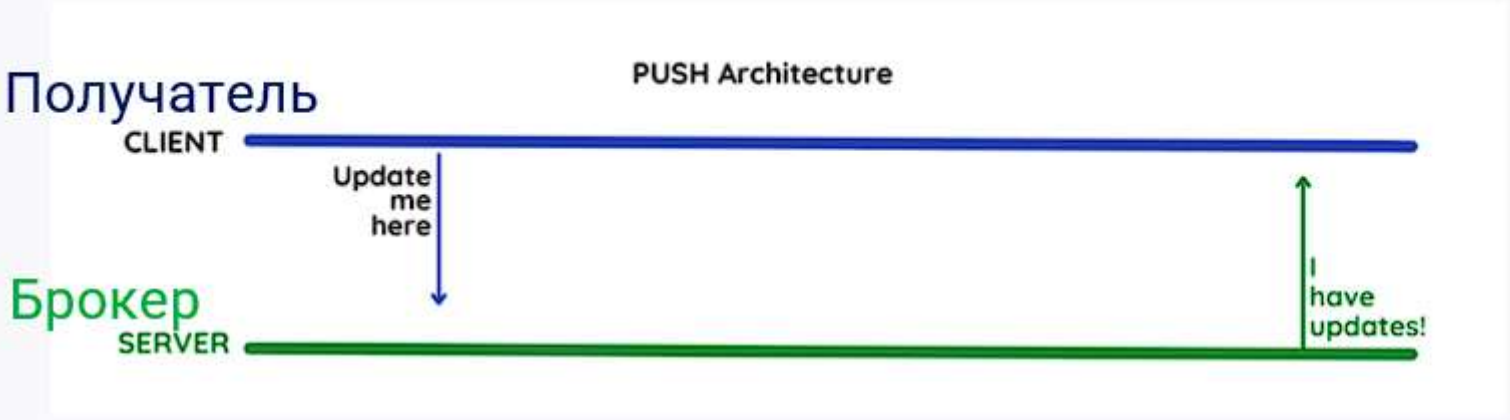


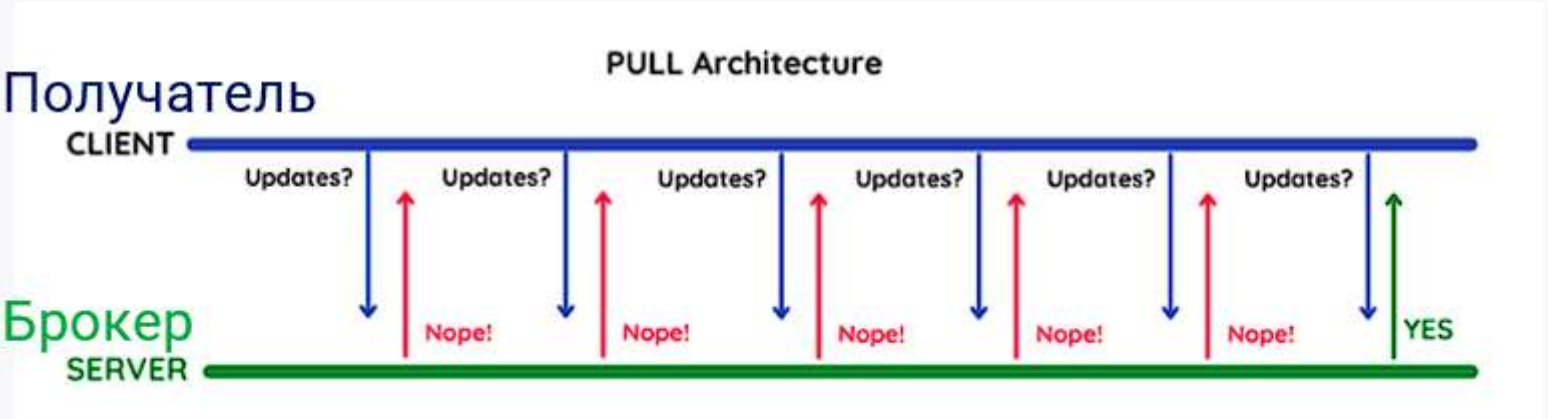
Практика



Kafka

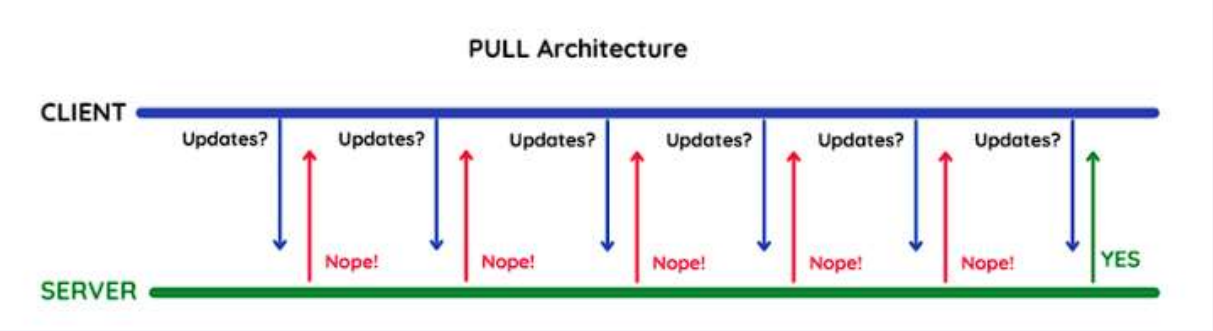
PUSH и PULL архитектуры брокеров





Kafka PULL - модель

Почему выбрана PULL-модель

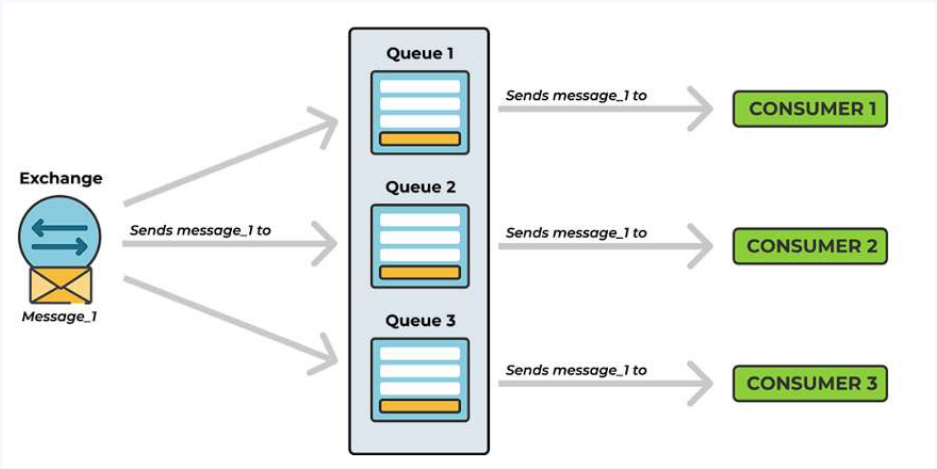


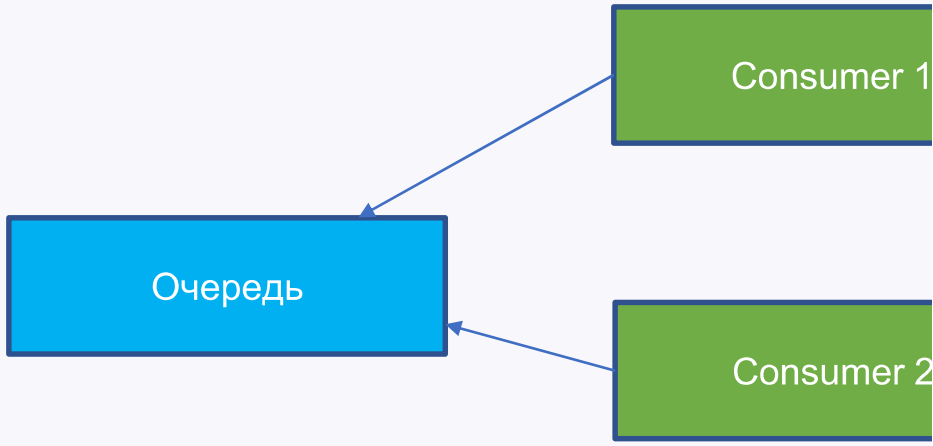
1. Есть консьюмеры быстрые, а есть медленные. PULL-модель позволяет всем обрабатывать сообщения по мере своих возможностей
2. Механизм подтверждения получения (ask) подразумевает наличие статусов сообщения и их обработку, а это накладные расходы

Недостаток PULL в том что потребитель вынужден постоянно запрашивать сообщен
компенсируется наличием периода ожидания сообщения.

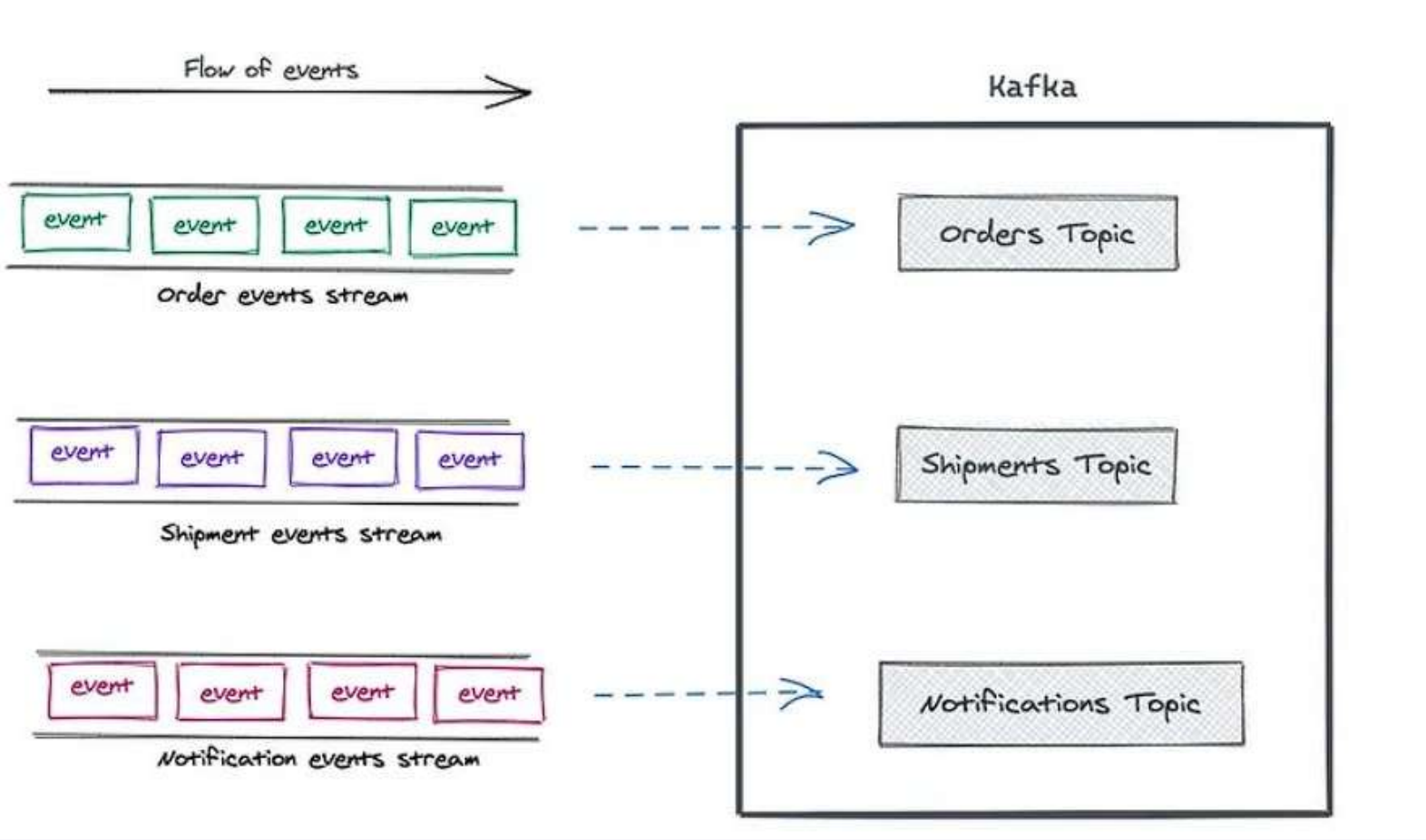
В Kafka сообщения не клонируются

В Kafka под каждого консьюмера не создается очередь (как в RMQ)



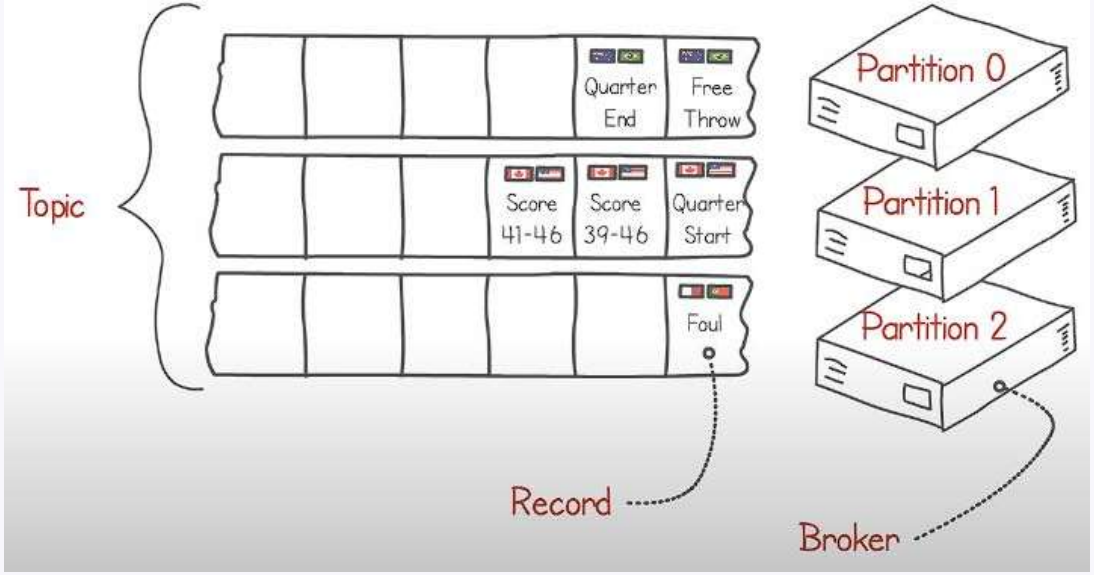


Topics



Topic – логический раздел, соответствующий определенному типу сообщений

Partitions

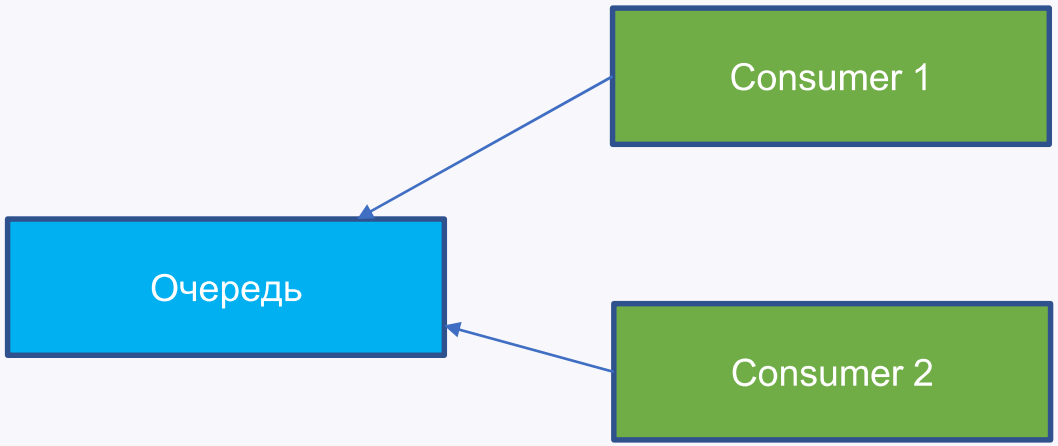


Partition – физический подраздел топика, соответствующий определенному подтипу сообщений и (или) хранящийся на отдельном сервере

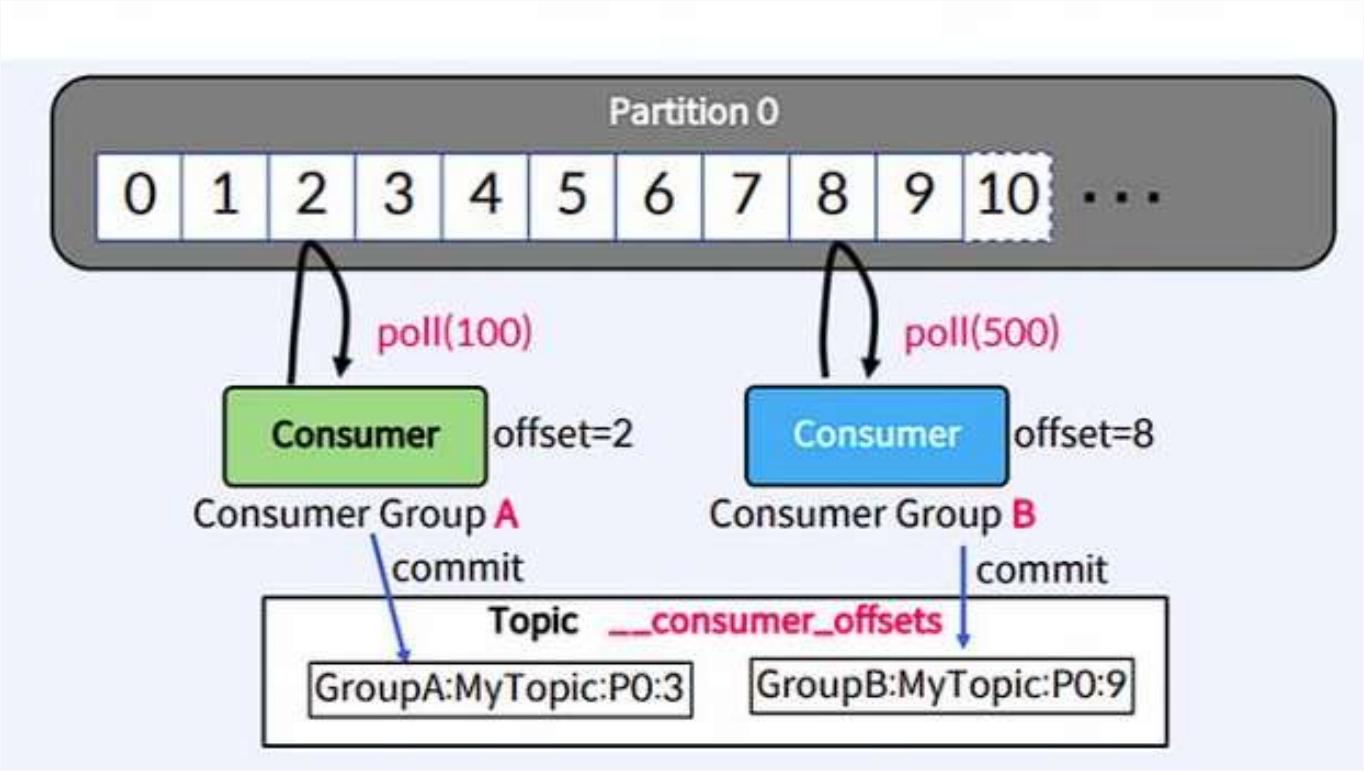
В пределах Partition гарантируется порядок обработки

Чтение сообщений из очереди

Как консьюмер понимает, с какого сообщения начать чтение?

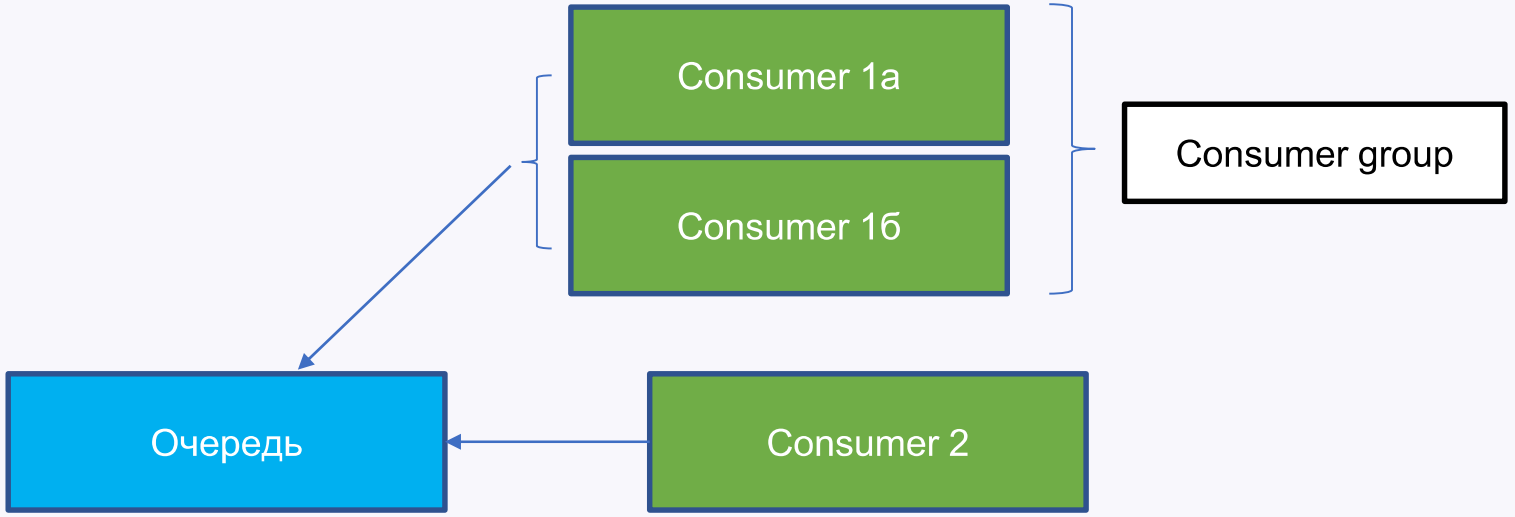


Offset



Offset показывает, с какой позиции данный Consumer Group должен читать сообщения

Consumer group



Консьюмеры с единым Consumer group действуют как реплики одного консьюмера

Kafka










Masstransit

Masstransit как удобная надстройка для Rabbi

nuget:

MassTransit

MassTransit.RabbitMQ



https://masstransit-project.com/getting-started/

Способы передачи сообщений в Masstransit

Три способа передачи сообщений

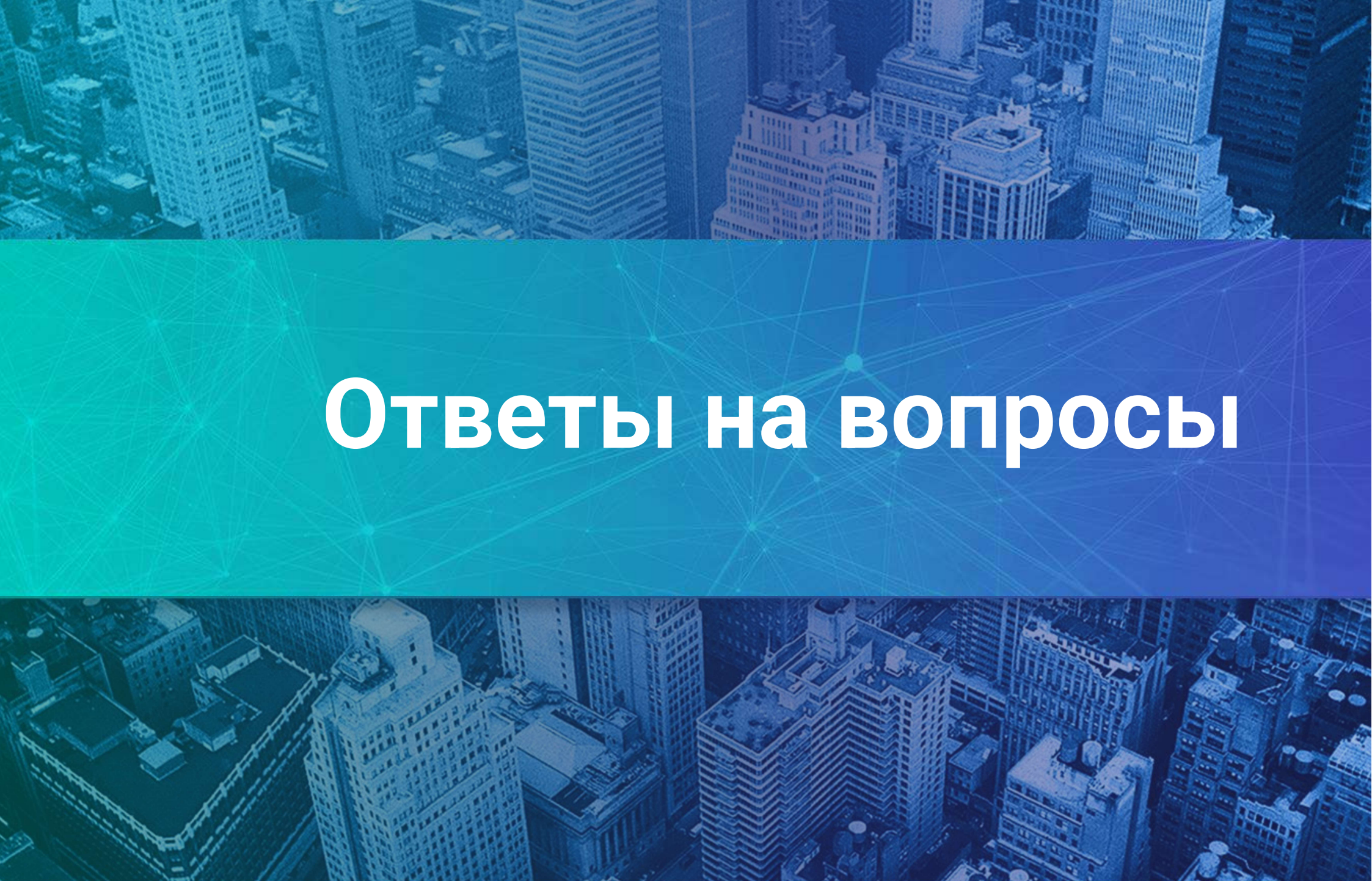
1. Publish – асинхронное, его получает тот, кто подписан на данный **тип сообщений**
2. Send – асинхронное, его получает только тот, кто подписан на очередь с **данным наименованием**
3. Request/Response – синхронное, его получает тот, кто подписан на данный тип сообщений

Механизм повторной обработки (Retry) сообщений в Masstransit

```
e.UseMessageRetry(r =>
{
    r.Handle<ArgumentNullException>();
    r.Ignore(typeof(InvalidOperationException), typeof(InvalidCastException));
    r.Ignore<ArgumentException>(t => t.ParamName == "orderTotal");
});
```

События, не обработанные повторно, попадают в **Fault** - очереди

Policy	Description
None	No retry
Immediate	Retry immediately, up to the retry limit
Interval	Retry after a fixed delay, up to the retry limit
Intervals	Retry after a delay, for each interval specified
Exponential	Retry after an exponentially increasing delay, up to th
Incremental	Retry after a steadily increasing delay, up to the retry



Ответы на вопросы

Список материалов для изучения

1. Об устройстве RMQ https://www.rabbitmq.com/tutorials/amqp-concepts.html
2. О библиотеке RMQ.Client https://www.rabbitmq.com/dotnet-api-guide.html
3. О кафке коротко https://medium.com/codex/asynchronous-communication-why-doeskafka-use-a-pull-based-message-consumer-442c19a70f58
4. Короткое но информативное видео о кафке https://youtu.be/Ch5VhJzaoaI?si=sNsRj72itoKdnCr_
5. Более длинное и сложное, но на русском https://youtu.be/-AZOi3kP9Js?si=b8wylQ4Rtp4RC-Ro
6. Библиотека Kafka.Confluent, примеры реализации. См.ReadMe снизу https://github.com/confluentinc/confluent-kafka-dotnet